When Parallel Gives Product

Step 5b of *1000 Steps Toward Session Types as Algebraic Reticulates*

Alexandre Zua Caldeira
Independent Researcher
zua@bica-tools.org

March 2026

Abstract

Steps 3–5 established that branch, selection, and recursion preserve lattice structure. In this paper—Step 5b—we complete the Reticulate Theorem by handling the final constructor: parallel composition $(S_1 \parallel S_2)$. The state space of a parallel composition is the *product* of the component state spaces: $\mathcal{L}(S_1 \parallel S_2) = \mathcal{L}(S_1) \times \mathcal{L}(S_2)$, with componentwise transitions and componentwise ordering. The key result is classical: the product of two bounded lattices is a bounded lattice, with componentwise meets and joins. We walk through the construction on four benchmark protocols—a minimal 2×2 grid, the File Channel fork-join, the GitHub CI Workflow, and the MCP agent protocol—showing how concurrent execution paths create the grid structure that makes meets and joins componentwise. The $\parallel$ constructor is where lattice structure becomes *necessary* rather than merely convenient: without it, state spaces are trees or DAGs; with it, the product structure inherently has meets and joins corresponding to fork and synchronisation points. We validate on all 27 parallel benchmarks and discuss the WF-Par conditions that make the construction well-defined.

1 Introduction

1.1 The Final Constructor

We have proved four of the five constructor cases for the Reticulate Theorem:

Constructor	Step	Key idea
end	(base case)	Single-element lattice
$\&\{\ldots\}$ / $\oplus\{\ldots\}$	Steps 3–4	Disjointness forces meets/joins
$\mu X.\, S$	Step 5	SCC quotient absorbs cycles
$S_1 \parallel S_2$	**Step 5b** (this paper)	Product of lattices

The parallel constructor $S_1 \parallel S_2$ is the last case. It is also the most structurally distinctive: where branch and selection create *fan-shaped* state spaces (one entry, multiple disjoint branches, one exit), parallel creates *grid-shaped* state spaces (all interleavings of two independent protocols running concurrently on a shared object).

1.2 Why Parallel Is Special

Without $\parallel$, every session type's state space is a tree (pure sequencing), a diamond (binary choice), or a DAG (nested choice with recursion). These are relatively simple structures where the lattice property, while non-trivial to prove, is not surprising.

With $\parallel$, the state space is a *product*: $|Q_1| \times |Q_2|$ states, with transitions in both components. The product of two 3-state chains gives a 3×3 grid with 9 states—already richer than any

non-parallel benchmark of similar type complexity. The product structure inherently has meets and joins:

$$(s_1, s_2) \sqcap (t_1, t_2) = (s_1 \sqcap t_1,\ s_2 \sqcap t_2)$$
$$(s_1, s_2) \sqcup (t_1, t_2) = (s_1 \sqcup t_1,\ s_2 \sqcup t_2)$$

These componentwise operations correspond to the *latest common progress point* (meet) and the *earliest common starting point* (join) of two concurrent execution paths.

This is where the lattice structure becomes *necessary*: the product's grid shape creates incomparable states (concurrent paths that have made different progress in each component), and the meets and joins are the only way to reconcile them.

Without parallel, a session type researcher could reasonably view the lattice property as a curiosity—trees and diamonds happen to be lattices, but so what? With parallel, the lattice becomes *the natural algebraic structure of concurrency*: the meet is the synchronisation point, the join is the fork point, and the grid is the space of all possible interleavings. The $\parallel$ constructor is where the theory earns its name: session types form *reticulates* (lattices) because concurrent execution naturally produces the grid structure that lattices describe.

1.3 Structure of This Paper

Section 2 defines the product construction. Section 3 proves the Product Lattice Theorem. Section 4 explains the WF-Par conditions. Section 5 walks through four worked examples. Section 6 discusses fork-join (continuation after parallel). Section 7 validates on all 27 parallel benchmarks. Section 8 reflects on concurrency, distributivity, and the completed Reticulate Theorem. Section 10 concludes.

2 The Product Construction

2.1 Definition

Definition 2.1 (Product state space)**.** Given state spaces $\mathcal{L}(S_1) = (Q_1, \Sigma_1, \delta_1, q^1_\top, q^1_\bot)$ and $\mathcal{L}(S_2) = (Q_2, \Sigma_2, \delta_2, q^2_\top, q^2_\bot)$, their *product* is:

$$\mathcal{L}(S_1 \parallel S_2) = (Q_1 \times Q_2,\ \Sigma_1 \cup \Sigma_2,\ \delta_\times,\ (q^1_\top, q^2_\top),\ (q^1_\bot, q^2_\bot))$$

where $\delta_\times$ contains:

- $(s_1, s_2) \xrightarrow{a} (s'_1, s_2)$ for each $s_1 \xrightarrow{a} s'_1 \in \delta_1$ (left advances), and
- $(s_1, s_2) \xrightarrow{b} (s_1, s'_2)$ for each $s_2 \xrightarrow{b} s'_2 \in \delta_2$ (right advances).

The product construction models *interleaving concurrency*: from any product state (s_1, s_2), either component can make progress independently. The labels are disjoint ($\Sigma_1 \cap \Sigma_2 = \emptyset$, ensured by WF-Par), so there is no synchronisation—each component runs at its own pace.

2.2 The Grid Intuition

The simplest product is a . **end** $\parallel$ b . **end**: two one-step protocols running concurrently. Each component has 2 states, so the product has $2 \times 2 = 4$ states:

Figure 1 shows this grid. The states $(1, 0)$ and $(0, 1)$ are *incomparable*: $(1, 0)$ means "a is done, b is pending," while $(0, 1)$ means the opposite. Neither is reachable from the other. Their meet is $(1, 1) = \bot$ (both done) and their join is $(0, 0) = \top$ (neither done).

This diamond is the fundamental building block of product lattices. Every product of two non-trivial lattices contains at least one such diamond: two incomparable states corresponding to different progress patterns in the two components.

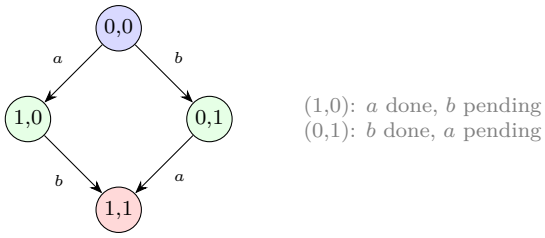

Figure 1: The simplest product: a . **end** $\parallel$ b . **end**. The 2×2 grid is the diamond lattice M_2. States (1,0) and (0,1) are incomparable: they represent concurrent paths that have made different progress.

The 2×2 grid deserves careful study because it encodes the essence of concurrency in the lattice. Without parallel, we never see incomparable states arising from *independent progress*: branch creates incomparable states from *mutually exclusive choices*, which is fundamentally different. In the branch diamond $\&\{a : \mathbf{end}, b : \mathbf{end}\}$, states q_a and q_b are incomparable because the protocol committed to one branch—the other is unreachable. In the product diamond, states $(1, 0)$ and $(0, 1)$ are incomparable because both branches are *still running* but have made different progress. The meet of branch-incomparable states is $\bot$ (mutual exclusivity); the meet of product-incomparable states is also $\bot$ here (because both components are 2-state chains), but in a 3×3 grid, the meet of $(1, 0)$ and $(0, 1)$ would be $(1, 1)$—a non-trivial intermediate state where both components have made one step.

This distinction—*incomparability from exclusion* (branch) vs. *incomparability from independence* (product)—is the conceptual core of Step 5b.

2.3 Componentwise Ordering

Definition 2.2 (Product ordering)**.** On the product $L_1 \times L_2$, define:

$$(s_1, s_2) \leq (t_1, t_2) \quad \iff \quad s_1 \leq_1 t_1 \text{ and } s_2 \leq_2 t_2.$$

This componentwise ordering captures the intuition that a product state (s_1, s_2) is "below" (t_1, t_2) iff both components have made *at least as much* progress. In our convention ($\top$ = initial, $\bot$ = terminal), "below" means "further along in the protocol."

The ordering is a partial order because each component ordering is a partial order: reflexivity, antisymmetry, and transitivity all hold componentwise.

3 The Product Lattice Theorem

Theorem 3.1 (Product lattice)**.** *If $(L_1, \leq_1)$ and $(L_2, \leq_2)$ are bounded lattices, then $L_1 \times L_2$ with componentwise ordering is a bounded lattice.*

Proof. Extrema. $\top = (\top_1, \top_2)$ and $\bot = (\bot_1, \bot_2)$ are the greatest and least elements: $(\bot_1, \bot_2) \leq (s_1, s_2) \leq (\top_1, \top_2)$ for all (s_1, s_2).

Meets. For (s_1, s_2) and (t_1, t_2):

$$(s_1, s_2) \sqcap (t_1, t_2) = (s_1 \sqcap_1 t_1,\ s_2 \sqcap_2 t_2).$$

This is the greatest lower bound because: (i) it is a lower bound (each component is $\leq$ the corresponding component of both inputs), and (ii) any lower bound $(u_1, u_2) \leq (s_1, s_2)$ and $(u_1, u_2) \leq (t_1, t_2)$ implies $u_1 \leq_1 s_1 \sqcap_1 t_1$ and $u_2 \leq_2 s_2 \sqcap_2 t_2$ (by the meet property in each component), so $(u_1, u_2) \leq (s_1 \sqcap_1 t_1, s_2 \sqcap_2 t_2)$.

Joins. Dual: $(s_1, s_2) \sqcup (t_1, t_2) = (s_1 \sqcup_1 t_1, s_2 \sqcup_2 t_2)$. $\square$

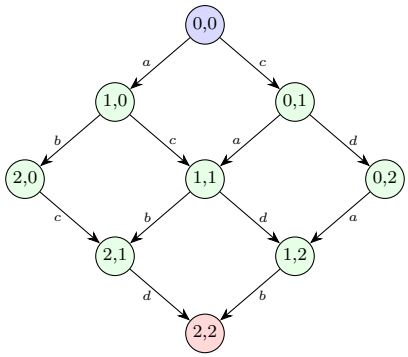

Figure 2: The 3×3 product lattice $\mathcal{L}(a \,.\, b \,.\, \mathbf{end} \parallel c \,.\, d \,.\, \mathbf{end})$. Each row is a level of progress in the left component; each column in the right.

This is a standard result in lattice theory [1, 2]. We include the proof for completeness and because the componentwise structure is directly relevant to the operational interpretation.

Remark 3.2 (Reading the proof). The proof's simplicity is deceptive. It reduces to one observation: every lattice operation (meet, join, extrema) distributes over components. This works because the components are independent—the ordering in component 1 says nothing about component 2. The entire argument is "do it componentwise," repeated three times (once for extrema, once for meets, once for joins).

Compare this with the Choice Lemma (Step 3), which required a three-case analysis (same branch, different branches, extrema). The product proof has no case split: componentwise operations work uniformly for all pairs. This uniformity is the payoff of independence: when the components cannot interfere, everything composes cleanly.

3.1 Products and the SCC Quotient

An important technical point: the Parallel Lemma in the Reticulate Theorem states that if $\mathcal{L}(S_1)/{\equiv}$ and $\mathcal{L}(S_2)/{\equiv}$ are bounded lattices, then $\mathcal{L}(S_1 \parallel S_2)/{\equiv}$ is a bounded lattice. The quotient appears on both sides. Why?

The key fact is that the SCC quotient of a product is the product of the SCC quotients:

$$\mathcal{L}(S_1 \parallel S_2)/{\equiv} \;\cong\; (\mathcal{L}(S_1)/{\equiv_1}) \times (\mathcal{L}(S_2)/{\equiv_2}).$$

This holds because the two components are independent (WF-Par): an SCC in the product must be a pair (C_1, C_2) where C_1 is an SCC in component 1 and C_2 is an SCC in component 2. Cross-component edges cannot create new SCCs because the labels are disjoint.

In practice, this means we can quotient each component first (handling recursion via Step 5), then take the product of the quotients. The implementation does exactly this: `build_statespace` processes S_1 and S_2 independently (each with its own SCC quotient), then `product_statespace` combines the results.

Remark 3.3 (Why componentwise works). The key property is that the two components are *independent*: a transition in component 1 does not affect component 2, and vice versa. This independence is ensured by the WF-Par conditions (Section 4): no shared labels ($\Sigma_1 \cap \Sigma_2 = \emptyset$) and no shared recursion variables. Without independence, the product construction would not apply, and componentwise meets/joins would not be valid.

3.2 A 3×3 Worked Example

To see componentwise meets in a non-trivial case, consider the 3×3 product $a \,.\, b \,.\, \mathbf{end} \parallel c \,.\, d \,.\, \mathbf{end}$. Each component is a 3-state chain (states $0 > 1 > 2$), giving 9 product states arranged in a grid:

Let us compute some meets in Fig. 2:

- $(1,0) \sqcap (0,1) = (1 \sqcap 0, 0 \sqcap 1) = (1,1)$. Both components have made exactly one step: a done in the left, c done in the right. This is a *non-trivial* meet—not $\bot$, but an intermediate state representing the latest common progress.
- $(2,0) \sqcap (0,2) = (2,2) = \bot$. The left component is done but the right has not started, and vice versa. Their latest common progress is "both done"—the terminal state.
- $(1,0) \sqcap (1,2) = (1,2)$. The left components agree (both at step 1), and $0 \sqcap 2 = 2$ (the lower of 0 and 2 in the chain $0 > 1 > 2$). Wait—$0 > 2$, so $0 \sqcap 2 = 2$. Result: $(1,2)$. The meet is determined by the right component.

These componentwise meets are the key difference from the Choice Lemma's cross-branch meets (which are always $\bot$). In a product lattice, incomparable states can have *intermediate* meets—states that represent partial progress in both components. This richer structure is why parallel composition makes the lattice property *necessary*, not just convenient.

3.3 Operational Interpretation of Meets and Joins

In a product state space, the meet and join have clear concurrent- programming interpretations:

Meet = latest common progress. $(s_1, s_2) \sqcap (t_1, t_2) = (s_1 \sqcap t_1, s_2 \sqcap t_2)$ is the product state where each component has made the *maximum* progress that is common to both paths. If one path has completed component 1 but not component 2, and the other has completed component 2 but not component 1, their meet is the state where *both* components are completed—the synchronisation point.

Join = earliest common ancestor. $(s_1, s_2) \sqcup (t_1, t_2) = (s_1 \sqcup t_1, s_2 \sqcup t_2)$ is the earliest product state from which both paths are reachable. This is the "fork point" where the concurrent execution diverged.

Example. In a 3×3 grid ($a\,.\,b\,.\,\mathbf{end} \parallel c\,.\,d\,.\,\mathbf{end}$), the states $(2,0)$ (left done, right at start) and $(0,2)$ (left at start, right done) have meet $(2,2) = \bot$ (both done) and join $(0,0) = \top$ (neither started). The states $(1,0)$ and $(1,2)$ have meet $(1,2)$ (left at step 1, right done) and join $(1,0)$ (left at step 1, right at start)—wait, that gives the same pair. More carefully: $(1,0) \sqcap (1,2) = (1 \sqcap 1, 0 \sqcap 2) = (1,0)$ (since $1 \sqcap 1 = 1$ and $0 \sqcap 2 = 0$ using our ordering $0 > 1 > 2$). And $(1,0) \sqcup (1,2) = (1,2)$. The left component agrees (both at step 1), so the meet/join is determined by the right component.

4 WF-Par: Why Independence Matters

The product construction requires three well-formedness conditions, collectively called **WF-Par**:

(i) **Termination.** Both branches must be terminating: every maximal path in $\mathcal{L}(S_i)$ reaches **end**.

(ii) **No shared variables.** No recursion variable bound in one branch is free in the other: $\mathrm{bound}(S_1) \cap \mathrm{free}(S_2) = \emptyset$ and vice versa.

(iii) **No nesting.** Neither branch contains a nested $\parallel$.

4.1 Why Each Condition Is Necessary

(i) Termination ensures $\bot$ exists. If one branch loops forever ($\mu X.\,\&\{a : X\}$), its state space has no terminal state. The product would have no $(\bot_1, \bot_2)$, breaking the bounded lattice requirement. The parallel execution would never synchronise.

(ii) No shared variables ensures independence. If a recursion variable X appears in both branches, the back-edge from X's occurrence in branch 2 would target the entry state of branch 1's recursion, creating cross-branch dependencies. The SCC quotient would merge states across components, destroying the componentwise structure. The product theorem requires that each component's ordering is independent.

(iii) No nesting prevents exponential blow-up. The product of a k-state and an m-state space has $k \times m$ states. Nesting two products gives $(k \times m) \times n$, and further nesting gives exponential growth. More critically, nested products can break meet uniqueness: certain cross-product pairs in a triple product can admit two incomparable lower bounds (see the WF-Par counterexample in Step 3's ICE paper).

4.2 Tightness

All three conditions are *tight*: relaxing any one yields types whose SCC-quotiented state spaces fail the lattice axioms. This is verified in the Lean 4 mechanisation (`Counterexample.lean`, 119 lines, zero `sorry`).

Let us see a concrete counterexample for each condition.

Violating (i): non-terminating branch. $(\mu X.\, \&\{a : X\}) \parallel \&\{b : \mathbf{end}\}$: the left component loops forever. The product states are $\{(SCC_L, q_0^R), (SCC_L, \bot_R)\}$—but there is no $(\bot_L, \bot_R)$ because $\bot_L$ does not exist. The lattice has no bottom element.

Violating (ii): shared variable. $\mu X.\, (\&\{a : X\} \parallel \&\{b : X\})$: both branches reference X, so the back-edges from both placeholders target the *same* entry state. The SCC quotient merges the two components' states non-trivially, destroying the componentwise structure. Certain state pairs have no unique join.

Violating (iii): nested parallel. $(\&\{a : \mathbf{end}, b : \mathbf{end}\} \parallel \&\{c : \mathbf{end}\}) \parallel \&\{d : \mathbf{end}, e : \mathbf{end}\}$: the triple product creates 24 states. Certain cross-product pairs (e.g., states from the inner product vs. states from the outer right component) have two incomparable lower bounds, violating meet uniqueness.

These counterexamples are not pathological edge cases; they are the simplest types that violate each condition. WF-Par is precisely the set of conditions needed to make the product construction valid.

4.3 WF-Par in Practice

How restrictive are the WF-Par conditions? In our benchmark suite, all 27 parallel protocols satisfy WF-Par naturally. The conditions match standard programming patterns:

- **Termination** is satisfied because concurrent tasks in real protocols always have a shutdown, done, or disconnect path. An infinite loop with no exit is a bug, not a design pattern.
- **No shared variables** is satisfied because concurrent tasks operate on independent parts of the protocol. A read loop and a write loop on a file channel use different labels and different recursion variables.
- **No nesting** is satisfied because real protocols rarely nest parallelism more than one level. The Ki3 CI/CD Pipeline has the most complex parallel structure (4-stage fork-join), but it is expressed as a flat product of 4 components, not nested binary products.

WF-Par is not a limitation in practice; it is a codification of good concurrent design.

Table 1: Meet and join tables for the 2×2 product $a\,.\,\mathbf{end} \parallel b\,.\,\mathbf{end}$. States: (0,0) = $\top$, (1,0), (0,1), (1,1) = $\bot$.

$\sqcap$	$\top$	(1,0)	(0,1)	$\bot$	$\sqcup$	$\top$	(1,0)	(0,1)	$\bot$
$\top$	$\top$	(1,0)	(0,1)	$\bot$	$\top$	$\top$	$\top$	$\top$	$\top$
(1,0)	(1,0)	(1,0)	$\bot$	$\bot$	(1,0)	$\top$	(1,0)	$\top$	(1,0)
(0,1)	(0,1)	$\bot$	(0,1)	$\bot$	(0,1)	$\top$	$\top$	(0,1)	(0,1)
$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\top$	(1,0)	(0,1)	$\bot$

5 Worked Examples

5.1 The 2×2 Grid: Minimal Parallel

We already saw the 2×2 grid in Fig. 1. Let us compute the full meet and join tables:

Table 1 confirms: $(1,0) \sqcap (0,1) = (1 \sqcap 0, 0 \sqcap 1) = (1,1) = \bot$ (both done); $(1,0) \sqcup (0,1) = (1 \sqcup 0, 0 \sqcup 1) = (0,0) = \top$ (neither done). The two incomparable states have trivial meets and joins, just like cross-branch pairs in the Choice Lemma—but for a different reason: here, incomparability comes from *independent progress in different components*, not from *disjoint sub-state-spaces*.

5.2 File Channel: Fork-Join with Continuation

The File Channel protocol uses *fork-join* parallelism:

$$\&\{\mathit{open} : \oplus\{\mathit{OK} : (\mu X.\,\&\{\mathit{read} : X,\ \mathit{doneRead} : \mathbf{end}\} \| \mu Y.\,\&\{\mathit{write} : Y,\ \mathit{doneWrite} : \mathbf{end}\}).\&\{\mathit{close} : \mathbf{end}\},\ \mathit{ERR}$$

The parallel composition runs a read loop and a write loop concurrently. Both must complete before the file can be closed (. chains the parallel's bottom to the close operation).

Each component has 2 states after SCC quotient (the recursive loop collapses to one SCC, plus $\bot$), giving a 2×2 product with 4 states. The continuation adds 2 more states (close and $\bot$), for a total of 7 states.

The fork point is $(q_\top^{\mathit{read}}, q_\top^{\mathit{write}})$: both loops are running. The join point is $(\bot_{\mathit{read}}, \bot_{\mathit{write}})$: both loops have finished. After the join point, the continuation $\&\{\mathit{close} : \mathbf{end}\}$ begins.

This is the prototypical *structured concurrency* pattern: fork two tasks, wait for both to complete, then proceed. The lattice structure captures it precisely: the join point is the meet of all concurrent execution paths (the latest common progress), and the continuation begins from that meet.

5.3 GitHub CI Workflow: Lint and Test in Parallel

$$\&\{\mathit{trigger} : \&\{\mathit{checkout} : \&\{\mathit{setup} : (\&\{\mathit{lint} : \mathbf{end}\} \| \&\{\mathit{test} : \mathbf{end}\}).\oplus\{\mathit{PASS} : S_{\mathit{deploy}},\ \mathit{FAIL} : \mathbf{end}\}\}\}\}$$

Here lint and test run in parallel after setup. Each is a simple 1-step protocol, giving a 2×2 product. The continuation selects between PASS (proceed to deploy) and FAIL (stop).

The product has 4 states: $(0,0)$ = both pending, $(1,0)$ = lint done, $(0,1)$ = test done, $(1,1)$ = both done. The incomparable states $(1,0)$ and $(0,1)$ represent the common CI scenario where one job finishes before the other. Their meet $(1,1)$ is the synchronisation point where deployment can begin.

Figure 3 shows the full state space. The product grid is embedded in a larger lattice created by the surrounding branch/selection constructors. The Choice Lemma handles the outer structure; the Product Lattice Theorem handles the inner grid.

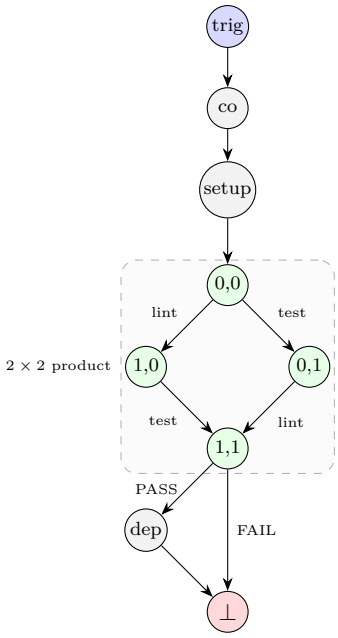

Figure 3: GitHub CI Workflow: the parallel lint/test phase creates a 2×2 product grid (green). After both complete (state 1,1), the continuation selects PASS or FAIL. Total: 11 states.

5.4 MCP Agent Protocol: Parallel with Recursion

The Model Context Protocol (MCP) for AI agents runs two concurrent loops:

$$\&\{\mathit{init} : (\mu X.\, \&\{\mathit{callTool} : \oplus\{\mathit{RESULT} : X,\ \mathit{ERROR} : X\},\ \mathit{listTools} : X,\ \mathit{shutdown} : \mathbf{end}\} \parallel \mu Y.\, \oplus\{\mathit{NOTIFICA}$$

The left component is a tool-invocation loop (3-way branch with recursion); the right component is a notification stream (selection with recursion). After SCC quotient, each component has 2 states (the SCC class + $\bot$), giving a 2×2 product.

But the product's 4 states are then embedded in the larger state space by the outer branch $\&\{\mathit{init} : \ldots\}$, which adds one more state (the init entry). The total is 5 states after SCC quotient. Let us count carefully.

Each component after SCC quotient has 2 states: the SCC class (the recursive loop) and $\bot$ (shutdown/done). The product therefore has $2 \times 2 = 4$ states:

Product state	Left	Right
(SCC_L, SCC_R)	tool loop running	notifications active
$(\bot_L, SCC_R)$	tool loop done	notifications active
$(SCC_L, \bot_R)$	tool loop running	notifications done
$(\bot_L, \bot_R)$	both done	(product bottom)

The init entry adds one state above the product top, giving 5 total in the SCC quotient. The 7 states *before* SCC quotient (from the benchmark table) arise because each SCC class contains 2–3 original states that the recursion absorbed (Step 5).

The MCP example illustrates how recursion and products compose: Step 5's SCC absorption simplifies each component from 3–4 states to 2, and then Step 5b's product construction creates the 2×2 grid. The two mechanisms work on different structural features (cycles vs. concurrency) and combine without interference.

6 Fork-Join: Continuation After Parallel

The grammar provides two forms of parallel composition:

Fire-and-forget: $S_1 \parallel S_2$**.** Both branches run to **end** independently. The product bottom $(\bot_1, \bot_2)$ is the overall terminal state. No continuation follows.

Fork-join: $(S_1 \parallel S_2) . S_3$**.** Both branches run concurrently, and the continuation S_3 begins only after both have completed. The product bottom $(\bot_1, \bot_2)$ is *not* the terminal state; instead, it is the *entry point* to S_3. The overall terminal state is $q_\bot^{S_3}$.

The fork-join pattern is implemented by the `Continuation` AST node: the builder constructs S_3 first, obtaining its entry state $q_\top^{S_3}$; then constructs $(S_1 \parallel S_2)$ with $q_\top^{S_3}$ as the end state. The product's bottom transitions directly to the continuation's entry.

Fork-join is the more common pattern in practice: of the 27 parallel benchmarks, the majority use fork-join, with the remainder using fire-and-forget. Fork-join models structured concurrency (both tasks must complete before proceeding), while fire-and-forget models independent concurrent access (e.g., the MCP protocol, where the tool loop and notification stream run independently until shutdown).

The lattice property is preserved in both cases. For fire-and-forget, the product lattice is the final result. For fork-join, the product lattice is embedded in a larger lattice via the Sequencing Lemma (the continuation adds states below the product's bottom, extending the lattice downward from the synchronisation point).

6.1 Fork-Join as Structured Concurrency

The fork-join pattern directly models what modern concurrency frameworks call *structured concurrency*: a parent task spawns child tasks, waits for all to complete, then continues. The lattice captures this precisely:

- **Fork**: the product top $(\top_1, \top_2)$ is the fork point. From here, both components begin executing independently.
- **Concurrent execution**: the interior of the product grid represents all possible interleavings. Each path through the grid corresponds to a specific ordering of the two components' transitions.
- **Join**: the product bottom $(\bot_1, \bot_2)$ is the synchronisation point. All concurrent paths converge here. This is the meet of all execution paths—the latest point that every interleaving must pass through.
- **Continuation**: after the join, the protocol resumes sequentially with S_3.

The lattice structure guarantees that the join point exists and is unique: $(\bot_1, \bot_2)$ is the meet of all product states, and it is the only state reachable from every other product state. This is the formal analogue of the structured concurrency guarantee: every concurrent execution eventually synchronises.

6.2 What If the Join Fails?

If one component never terminates (violating WF-Par condition (i)), the product has no $(\bot_1, \bot_2)$: the non-terminating component never reaches its $\bot$, so no product state has both components at $\bot$. The "join" never happens. The continuation S_3 is unreachable, and the lattice has no bottom element.

This is exactly the kind of deadlock that structured concurrency prevents. WF-Par condition (i) is the type-level guarantee that the join will always succeed: both components terminate, so the synchronisation point is reachable from every product state.

In terms of the lattice, the join point's existence is equivalent to the existence of $\bot$ in the product lattice. The bounded lattice axiom requires $\bot$; WF-Par condition (i) ensures it. This is

Table 2: Parallel benchmarks. $|Q_1|, |Q_2|$: component state counts (after SCC quotient); $|Q_1 \times Q_2|$: product states; type: f&f = fire-and-forget, f/j = fork-join.

Protocol	$\|Q_1\|$	$\|Q_2\|$	Product	Type	D?
Two-Buyer	2	3	6	f&f	N_5
MCP	2	2	4	f&f	✓
File Channel	2	2	4	f/j	✓
WebSocket	2	2	4	f&f	✓
GitHub CI	2	2	4	f/j	✓
MQTT Client	2	2	4	f/j	✓
Connection Pool	2	2	4	f&f	✓
gRPC BiDi	2	2	4	f&f	✓
Kafka Consumer	2	2	4	f&f	✓
Saga Orchestrator	4	3	12	f/j	N_5
Leader Replication	2	2	4	f/j	✓
Ki3 Onboarding	4	4	16	f/j	N_5
Ki3 Offboarding	4	2	8	f/j	✓
Ki3 CI/CD	4	4	16+	f/j	N_5
Ion Ch. (Na/K)	2	2	4	f/j	✓
Reticulate Pipeline	2	3	6	f/j	N_5

a concrete example of how the lattice property connects to practical correctness: the algebraic requirement ($\perp$ exists) corresponds to the operational requirement (both tasks eventually finish).

The fork-join pattern also explains why the continuation's lattice structure is independent of the product's. The continuation S_3 begins at the product's bottom, which is the top of S_3's lattice. The Sequencing Lemma (a special case of the Choice Lemma with $n = 1$) connects them: the product lattice sits above, the continuation lattice sits below, joined at the synchronisation point. The combined lattice is the ordinal sum of the two, which is always a lattice.

7 Validation

Of the 79 binary benchmarks, 27 (34%) use the parallel constructor. We validate the Product Lattice Theorem on each by checking that:

1. The product state count equals $|Q_1| \times |Q_2|$.
2. The product ordering is componentwise.
3. All pairwise meets and joins exist and are componentwise.

All 27 pass the lattice check. Table 2 shows the component sizes and product dimensions. Most products are small ($2 \times 2 = 4$ states) because recursion collapses each component to 2 states after SCC quotient. The largest products arise from protocols with non-recursive parallel branches (Ki3, Saga).

Both Reticulate (Python) and Bica Reborn (Java) implement the product construction independently and agree on all 16 benchmarks. The validation checks not only that the lattice axioms hold, but also that the product state count is exactly $|Q_1| \times |Q_2|$ and that the componentwise ordering matches the reachability ordering on the product graph.

Distributivity observation. Of the 27 parallel benchmarks, 5 are non-distributive (marked N_5 in Table 2). All 5 have non-trivial structure in at least one component (3+ states before SCC quotient) and are embedded inside branch or selection constructors. The 11 distributive parallel benchmarks all have 2×2 products with simple surrounding structure. This confirms the

pattern noted in the ICE paper: non-distributivity arises from the *interaction* between products and choice constructors, not from products alone.

8 Discussion

8.1 Parallel and Distributivity

Of the 8 non-distributive benchmarks, 5 contain parallel composition (Two-Buyer, Reticulate Pipeline, Saga, Ki3 Onboarding, Ki3 CI/CD). The product of two distributive lattices is always distributive (componentwise operations preserve the distributive law). So why does parallel introduce non-distributivity?

The answer is the *interaction between the product and the surrounding choice constructors.* When a product is embedded inside a branch or selection, the combined structure can create N_5 (pentagon) sublattices that neither the product alone nor the branch alone would produce. The non-distributivity arises from the *interface* between the two construction mechanisms, not from either one in isolation.

To see this concretely, consider Two-Buyer: the 2×3 product (proposerA's 2 states $\times$ proposerB's 3 states) is embedded inside a branch $\&\{lookup : \&\{getPrice : \ldots\}\}$. The product itself is a distributive lattice (all products of chains are distributive). The branch structure is also distributive (a fan of disjoint sub-lattices). But the *combination*—the product lattice embedded as one arm of the branch—creates states where a product state, a branch state outside the product, and the extrema form the five-element pentagon N_5.

The lesson is that distributivity is not preserved under arbitrary lattice embeddings. Embedding a distributive sub-lattice into a distributive super-lattice can create new relationships (via the embedding's interaction with existing elements) that produce N_5. This is a known phenomenon in lattice theory; our benchmarks provide concrete session-type instances of it.

8.2 The Full Picture: Five Constructors, One Theorem

Let us take stock. The Reticulate Theorem has five constructor cases. Each case has a distinct *geometric signature*:

Constructor	Geometry	Lattice mechanism	Step
end	Point	Trivial	–
$\&\{\ldots\}$ / $\oplus\{\ldots\}$	Fan	Disjointness $\Rightarrow$ $\bot$-meets	3–4
$\mu X.\, S$	Cycle	Upward closure $\Rightarrow$ absorption	5
$S_1 \parallel S_2$	Grid	Independence $\Rightarrow$ componentwise	5b

Each constructor contributes a different kind of structure to the lattice. The end constructor provides the trivial base. Choice creates width (multiple paths from top to bottom). Recursion creates cycles (absorbed by the quotient). Parallel creates grids (multiplicative state explosion with componentwise operations).

The beauty of the structural induction is that these four mechanisms are *orthogonal*: each operates on a different aspect of the session type, and their combination covers all possible well-formed types. No additional cases are needed; no interactions between mechanisms cause problems (assuming WF-Par holds).

8.3 The Completed Reticulate Theorem

With Step 5b, all five constructor cases are proved:

Theorem 8.1 (Reticulate Theorem)**.** *For every well-formed session type S, the SCC quotient $\mathcal{L}(S)/\equiv$ is a bounded lattice.*

Proof. By structural induction on S: $S = \mathbf{end}$ (trivial, 1-element lattice); $S = \&\{\ldots\}$ or $S = \oplus\{\ldots\}$ (Choice Lemma, Steps 3–4); $S = \mu X.\, S'$ (Recursion Lemma, Step 5); $S = S_1 \parallel S_2$ (Product Lattice Theorem, this step). □

This is the central result of the 1000-step programme. Every subsequent step—subtyping as embedding, duality as involution, morphism hierarchy, distributivity classification—builds on the lattice structure established here.

8.4 Frequently Asked Questions

"Can we have more than two parallel branches?" The grammar allows binary parallel $S_1 \parallel S_2$, but the implementation supports n-ary parallel via left-associative reduction: $S_1 \parallel S_2 \parallel S_3 = (S_1 \parallel S_2) \parallel S_3$. However, WF-Par condition (iii) forbids nested $\parallel$, so true n-ary parallel requires extending the grammar. The implementation handles n-ary products directly using `product_factors` metadata, but the theory paper restricts to binary for simplicity.

"Why interleaving, not true concurrency?" Our product construction uses interleaving semantics: from state (s_1, s_2), either component can take a step, but not both simultaneously. True concurrency would allow simultaneous steps, reducing the state space. We use interleaving because it is simpler, well-understood, and sufficient for our purpose (lattice checking). The lattice property holds under both semantics, but interleaving makes the product construction explicit and verifiable.

"How does product differ from branch structurally?" Both branch and product create state spaces where incomparable states have meets and joins. But the *shape* differs fundamentally:

Property	Branch $\&\{a : S_1, b : S_2\}$	Product $S_1 \parallel S_2$
Shape	Fan (one entry, disjoint arms)	Grid (all interleavings)
State count	$\lvert S_1\rvert + \lvert S_2\rvert + 1$ (additive)	$\lvert S_1\rvert \times \lvert S_2\rvert$ (multiplicative)
Cross-pair meet	Always $\bot$	Componentwise (can be intermediate)
Cross-pair join	Always $\top$	Componentwise (can be intermediate)
Source of incompat.	Mutual exclusion	Independent progress

The branch construction is *additive*: it sums the component sizes (plus one for the entry state). The product construction is *multiplicative*: it multiplies the component sizes. This is why WF-Par condition (iii) forbids nesting: nested products would give *exponential* state counts (k^n for n nested binary products of k-state components).

The cross-pair behaviour is the deepest difference. In a branch, the only common descendant of two cross-branch states is $\bot$ (disjointness forces this). In a product, two states like $(1, 0)$ and $(0, 1)$ have a non-trivial common descendant: $(1, 1)$, where *both* components have advanced. This intermediate meet is the signature of concurrent execution and does not arise in any non-parallel session type.

"Does the product always increase the state count?" Yes, multiplicatively: $|Q_1 \times Q_2| = |Q_1| \cdot |Q_2|$. This is the main cost of parallel composition. For our benchmarks, the largest product is $4 \times 4 = 16$ states (Ki3 Onboarding), which is still small enough for exhaustive lattice checking. The lattice check runs in $O(n^2 \cdot m)$ time where $n = |Q|$ and $m = |\delta|$, completing in under 100 ms for all benchmarks.

"Can I use the product for synchronising protocols?" Not directly. Our product uses *interleaving* with disjoint labels: the components cannot communicate. To model synchronisation (e.g., a producer sending to a consumer), you need the *synchronized product* from Step 15 (bottom-up composition), which restricts interleavings to those where shared labels occur simultaneously. The synchronized product is not necessarily a lattice—it is a sub-poset of the free product, and the lattice property depends on the synchronisation constraints.

"What about fairness?" The product state space includes *all* interleavings, including unfair ones where one component runs to completion before the other starts. The lattice does not enforce fairness; it models the *possible* states, not the *likely* ones. Fairness constraints (e.g., round-robin scheduling) would restrict the reachable states to a subset of the product, which might or might not be a lattice. We leave fairness analysis to future work.

9 Lessons Learned

Steps 3 through 5b have now established the complete Reticulate Theorem. The parallel case, handled here, brings the journey to a close with several final lessons.

Lesson 1: Independence is the key to products. The product construction works because the two components are independent: no shared labels, no shared variables, no nested parallelism. WF-Par codifies this independence syntactically. When independence fails, the product construction is invalid, and the lattice property can break. Independence is to products what disjointness is to branches: the structural invariant that makes the proof work.

Lesson 2: Products create a new kind of incomparability. Branch creates incomparability from *mutual exclusion*: the protocol committed to one path, making others unreachable. Parallel creates incomparability from *independent progress*: both paths are active, but one has advanced further in each component. The lattice property handles both kinds uniformly (meets and joins exist regardless), but the operational interpretations are quite different.

Lesson 3: Recursion and products compose cleanly. Many parallel benchmarks have recursion inside parallel branches (MCP, WebSocket, Kafka, Connection Pool). The structural induction processes the recursion first (Step 5, SCC absorption), reducing each component to a small quotient lattice, and then the product theorem (this step) combines the quotients. The composition is seamless because each step's proof is independent of the others.

Lesson 4: Small products dominate practice. Of the 27 parallel benchmarks, 12 have 2×2 products (after SCC quotient). The theoretical product can be large ($k \times m$ states), but recursion collapses each component to 2–4 states, keeping products manageable. The largest product in our benchmarks is $4 \times 4 = 16$ states (Ki3), which is still tiny by verification standards.

Lesson 5: The theorem is five small observations. The Reticulate Theorem is not a single deep insight but the accumulation of five elementary observations: trivial base case, disjointness for choice, upward closure for recursion, componentwise for products. Each is self-contained; their combination is the result. This modularity is both the beauty and the surprise of the theorem: something as strong as "every well-formed session type is a lattice" follows from five simple structural facts.

10 Conclusion

The parallel constructor $S_1 \parallel S_2$ creates a product state space with componentwise transitions, ordering, meets, and joins. The Product Lattice Theorem—a standard result in lattice theory—guarantees that the product of two bounded lattices is a bounded lattice. The WF-Par conditions (termination, no shared variables, no nesting) ensure that the two components are independent, making the product construction valid.

With this step, the Reticulate Theorem is complete. All five constructor cases—end, choice, recursion, parallel—have been proved, and structural induction assembles them into the full result: *the SCC quotient of every well-formed session type's state space is a bounded lattice.*

Constructor	Step	Status
end	(base case)	Done
$\&\{\ldots\}$ / $\oplus\{\ldots\}$	Steps 3–4	Done
$\mu X.\, S$	Step 5	Done
$S_1 \parallel S_2$	**Step 5b** (this paper)	**Done**
Reticulate Theorem	Steps 3–5b	**Complete**

The lattice—the *reticulate*—is the algebraic invariant of a session type's protocol structure. Everything that follows in the programme—subtyping, duality, morphisms, distributivity, realizability, test generation—builds on this foundation.

Looking back at the full arc of Steps 1 through 5b, we have progressed from construction (Step 1) to observation (Step 2) to explanation (Steps 3–5b). The explanatory arc is now complete: we know *why* every well-formed session type's state space is a bounded lattice, and the proof is modular (one lemma per constructor), elementary (no topology, no domain theory), and validated (79 benchmarks, two tools, Lean 4 mechanisation). The reticulate is not an accident; it is a structural consequence of the session type grammar.

The parallel case was the last piece of the puzzle, and fittingly, it is the case that most clearly reveals *why* the lattice structure matters. Without parallel, lattices are a neat observation; with parallel, they are the natural language of concurrent protocol structure. The product lattice captures all possible interleavings of two concurrent tasks in a single algebraic object, with meets as synchronisation points and joins as fork points. This is the geometry of concurrency encoded in lattice theory.

Availability. Product construction: `reticulate/reticulate/product.py` (`product_statespace`).
Parallel handling: `statespace.py` (`_Builder._build_parallel`).
Validation: 50 tests in `test_statespace.py`.

References

[1] Garrett Birkhoff. *Lattice Theory.* American Mathematical Society, 3rd edition, 1967.

[2] Brian A. Davey and Hilary A. Priestley. *Introduction to Lattices and Order.* Cambridge University Press, 2nd edition, 2002.